

INFORME DE JUSTIFICACIÓN DE PATRONES DE DISEÑO

1. Introducción

El presente documento justifica la arquitectura del diagrama de clases correspondiente al sistema de gestión de Sailor Scouts, misiones y combates. El modelado busca garantizar un código extensible, mantenible y alineado con los principios SOLID, aplicando patrones de diseño estructurales y de comportamiento, así como buenas prácticas en el modelado de bases de datos y relaciones de objetos.

2. Patrones de Diseño Aplicados

Patrón State (Estado)

- **Componentes implicados:** *Mision*, *EstadoMision* (*PENDIENTE*, *EN_CURSO*, *FINALIZADA*).
- **Justificación:** La clase *Mision* requiere cambiar su comportamiento interno dependiendo de su fase actual. Al encapsular las fases en la enumeración *EstadoMision*, se evita el uso de condicionales anidados complejos. Esto permite validar acciones de negocio según la etapa (por ejemplo, permitir agregar participantes solo si el estado es *PENDIENTE* o registrar el uso de ataques únicamente cuando esté *EN_CURSO*).

Patrón Strategy (Estrategia)

- **Componentes implicados:** *SailorScout*, *Ataque*, *Rango*.
- **Justificación:** Se encapsulan las habilidades (*Ataque*) y los niveles de poder/autoridad (*Rango*) fuera de la clase principal *SailorScout*. Esto permite desacoplar los comportamientos y atributos de combate, facilitando la adición de nuevos ataques o rangos sin necesidad de modificar el código de la Sailor Scout (Principio Abierto/Cerrado).

3. Patrones de Modelado Específicos (Trazabilidad y Auditoría)

Association Class / Object Versioning (Objetos de Transacción e Historial)

- **Componentes implicados:** *UsoAtaque*, *HistorialEquipo*, *HistorialRango*.
- **Justificación:**
 - **Relaciones N:M con contexto:** Una Sailor Scout utiliza múltiples ataques en múltiples misiones. La clase *UsoAtaque* actúa como entidad

intermedia para registrar los metadatos de esa interacción puntual (fecha, energiaConsumida).

- Trazabilidad temporal: Las clases **HistorialEquipo** e **HistorialRango** permiten mantener un registro auditable del cambio de estado y evolución del personaje a lo largo del tiempo (fechaInicio, fechaFin), en lugar de sobrescribir simplemente el valor actual.

4. Cumplimiento de Principios SOLID

- **Single Responsibility Principle (SRP):** Cada clase cumple una función clara. **SailorScout** gestiona los datos e interacciones de la entidad principal, **Mision** coordina el flujo del evento, y las clases de historial únicamente auditan cambios temporales.
- **Open/Closed Principle (OCP):** El sistema permite agregar nuevos tipos de enemigos, rangos o ataques sin alterar las clases centrales.
- **Liskov Substitution Principle (LSP):** La estructura desacoplada mediante agregaciones facilita la extensibilidad sin romper la compatibilidad con otras clases.

